

BACKEND DEVELOPMENT & DATABASES

Building Production-Ready APIs

with FastAPI, Databases, and Modern Backend Practices

Ishfaq Ahmad Dar

A Practical Guide to Real Backend Development

**Covering APIs, Authentication, Databases, ORM, NoSQL, Caching, and
Optimization**

First Edition

2026

© 2026 Ishfaq Ahmad Dar

All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without prior permission of the author.

□ About This Book

This book focuses on **real backend development**. It bridges the gap between theory and practical implementation by guiding you through building production-ready APIs, integrating databases, and optimizing performance.

It is designed for:

- Students preparing for backend developer roles
- Developers who know Python basics but want to build real systems
- Anyone aiming to become job-ready in backend development

□ What You Will Learn

- Building APIs using FastAPI and Flask
- Dependency injection and clean architecture
- Authentication and security (JWT, OAuth basics)
- SQL fundamentals and advanced queries
- Database design and transactions
- SQLAlchemy ORM for real applications
- NoSQL databases (MongoDB)
- Redis for caching and performance
- Backend optimization techniques

□ **How to Use This Book**

- Study **1 chapter per day**
- Build mini features alongside chapters
- Practice database queries regularly
- Apply concepts in real backend projects

□ **Goal of This Book**

To make you a **production-ready backend developer** capable of:

- Building scalable APIs
- Designing efficient databases
- Optimizing backend performance
- Handling real-world backend challenges

Build real backend systems with confidence. □

□ **BOOK 2: Backend Development & Databases**

□ **PART 1: FastAPI & Flask**

Chapter 1: FastAPI Basics

(Routing, Request/Response Models, Validation)

1. Introduction (Why This Matters in Backend)

Backend development is fundamentally about **handling requests and returning responses**. Every modern application—whether it’s a web app, mobile app, or microservice—relies on APIs.

□ **What Actually Happens in Backend?**

When a user performs an action (e.g., login):

1. Frontend sends a request → `/login`
2. Backend receives request
3. Backend validates data
4. Backend processes logic (DB, auth, etc.)
5. Backend returns response

□ **Why FastAPI?**

FastAPI is designed for **modern backend systems**:

- Built on **ASGI (Asynchronous Server Gateway Interface)**
- Supports **async programming (high performance)**
- Uses **type hints + Pydantic for validation**
- Automatically generates **interactive API docs**

□ Performance Insight

FastAPI is one of the fastest Python frameworks because:

- Uses **async I/O**
- Built on **Starlette + Uvicorn**
- Minimizes overhead

□ Comparable to Node.js and Go for I/O-heavy tasks

□ Real Industry Use

FastAPI is used for:

- Microservices
- Machine learning APIs
- Real-time systems
- High-performance APIs

□ Mastering FastAPI means:

- You can build **production-grade backend systems**
- You understand **modern backend architecture**
- You are **job-ready**

2. Core Concepts

2.1 What is an API? (Deep Understanding)

API = **Application Programming Interface**

It acts as a **contract** between:

- Client (frontend, mobile app)
- Server (backend)

□ Request-Response Cycle

Client → HTTP Request → Backend → HTTP Response → Client

□ HTTP Methods (Very Important)

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update data
DELETE	Remove data

□ Example:

GET /users → Fetch users

POST /users → Create user

□ Backend Insight:

- APIs must be **predictable**
- APIs must be **consistent**
- APIs must follow **standards (REST)**

2.2 Installing FastAPI (Proper Setup)

Install Required Packages:

```
pip install fastapi uvicorn
```

Why Uvicorn?

- ASGI server
- Runs FastAPI app
- Handles requests

Run Server:

```
uvicorn main:app --reload
```

Flags Explained:

- `main` → file name
- `app` → FastAPI instance
- `--reload` → auto-restart on changes

Production Note:

- In production, use:
 - Gunicorn + Uvicorn workers

2.3 Creating First FastAPI App (Deep Breakdown)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello World"}
```

□ Breakdown:

- `FastAPI()` → creates application instance
- `@app.get("/")` → defines route
- `home()` → request handler

□ Response Behavior:

- Python dict → automatically converted to JSON

□ Async Version:

```
@app.get("/")
async def home():
    return {"message": "Hello World"}
```

□ Use async when:

- Calling DB
- Calling APIs
- Performing I/O

2.4 Routing (Deep Understanding)

Routing maps **URL** → **function**.

□ GET Route

```
@app.get("/users")
def get_users():
    return ["Ali", "Sara"]
```

□ POST Route

```
@app.post("/users")
def create_user():
    return {"status": "created"}
```

□ Path Parameters

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"id": user_id}
```

Important:

- `user_id: int` → automatic validation
- If wrong type → FastAPI returns error

□ Query Parameters

```
@app.get("/search")
def search(q: str, limit: int = 10):
    return {"query": q, "limit": limit}
```

□ Multiple Parameters

```
@app.get("/items/{id}")
def get_item(id: int, sort: str = "asc"):
    return {"id": id, "sort": sort}
```

□ Backend Insight:

- Path params → identify resource

- Query params → filter/search

2.5 Request Body (Deep Understanding)

Request body contains **data sent by client**.

□ Using Pydantic Model

```
from pydantic import BaseModel
```

```
class User(BaseModel):  
    name: str  
    age: int
```

□ Endpoint

```
@app.post("/users")  
def create_user(user: User):  
    return user
```

□ What Happens Internally?

1. JSON → parsed
2. Validated by Pydantic
3. Converted to Python object

❑ Invalid Input Example:

```
{  
  "name": "Ali",  
  "age": "twenty"  
}
```

❑ FastAPI automatically returns:

- 422 error
- Validation message

❑ Backend Insight:

- Always validate input
- Never trust client data

2.6 Response Models (Deep Understanding)

Response models define **output schema**.

Example:

```
@app.get("/user", response_model=User)  
def get_user():  
    return {"name": "Ali", "age": 25}
```

□ **Why Important?**

- Ensures correct output
- Filters unwanted fields
- Improves API consistency

□ **Example (Filtering):**

```
@app.get("/user", response_model=User)
def get_user():
    return {"name": "Ali", "age": 25, "password": "123"}
```

□ **Output:**

```
{
  "name": "Ali",
  "age": 25
}
```

□ **Backend Insight:**

- Never expose sensitive data
- Always use response models

2.7 Validation (Advanced)

Using Field Constraints:

```
from pydantic import Field

class User(BaseModel):
    age: int = Field(gt=0, lt=120)
```

Other Validations:

```
name: str = Field(min_length=3, max_length=50)
```

Custom Validation:

```
from pydantic import validator

class User(BaseModel):
    name: str

    @validator("name")
    def validate_name(cls, v):
        if not v.isalpha():
            raise ValueError("Invalid name")
        return v
```

Backend Insight:

- Prevent invalid data early
- Reduce bugs

2.8 Automatic Documentation (Deep Use)

URLs:

- /docs → Swagger UI
- /redoc → Alternative docs

Features:

- Test API directly
- View schema
- Try requests

Backend Benefit:

- Saves development time
- No need to manually document APIs

2.9 Status Codes (Best Practices)

Example:

```
from fastapi import status

@app.post("/users", status_code=status.HTTP_201_CREATED)
def create_user():
    return {"status": "created"}
```

Common Codes:

Code	Meaning
200	Success
201	Created
400	Bad request
401	Unauthorized
404	Not found
500	Server error

Backend Insight:

- Always return correct status code
- Helps frontend understand response

3. Real-World Examples

Example 1: User API

```
@app.get("/users")
def get_users():
    return [{"id": 1, "name": "Ali"}]
```

Example 2: Create User

```
@app.post("/users")
def create_user(user: User):
    return {"message": "User created", "data": user}
```

Example 3: Search API

```
@app.get("/search")
def search(q: str, page: int = 1):
    return {"query": q, "page": page}
```

Example 4: Product Validation

```
class Product(BaseModel):
    price: float = Field(gt=0)
```

4. Code Examples (Important Patterns)

Pattern 1: Clean Route

```
@app.get("/items")
def get_items():
    return []
```

Pattern 2: Validation Layer

```
class Item(BaseModel):
    name: str
```

Pattern 3: Standard Response Format

```
return {  
    "status": "success",  
    "data": item  
}
```

Pattern 4: Error Handling

```
from fastapi import HTTPException  
  
@app.get("/items/{id}")  
def get_item(id: int):  
    if id < 0:  
        raise HTTPException(status_code=400, detail="Invalid  
ID")
```

5. Common Mistakes

✗ Not using Pydantic models

✗ Trusting client input

✗ Mixing business logic in routes

✗ Ignoring validation

✗ Not using async where needed

✗ Returning inconsistent responses

6. Interview Questions with Answers

Q1: What is FastAPI?

Modern Python framework for building APIs

Q2: Why FastAPI is fast?

Uses async + ASGI

Q3: What is routing?

Mapping URL → function

Q4: What is Pydantic?

Data validation library

Q5: What is request model?

Defines input schema

Q6: What is response model?

Defines output schema

Q7: Path vs Query parameters?

- Path → part of URL
- Query → filters

Q8: Why validation is important?

Prevents invalid data

7. Summary (Quick Revision)

- FastAPI → high-performance API framework
- API = request → response
- Routing:
 - GET
 - POST
- Parameters:

- Path
 - Query
 - Body
- Pydantic → validation
 - Response model → safe output
 - Status codes → correct communication

□ Master this chapter =

✓ Build real APIs

✓ Understand backend deeply

✓ Become job-ready □

Chapter 2: Dependency Injection

(Depends(), Database Sessions, Clean Architecture in FastAPI)

1. Introduction (Why This Matters in Backend)

As backend applications grow, complexity increases:

- Multiple routes
- Database connections
- Authentication logic

- Shared services

If everything is written inside route functions:

✗ Code becomes messy

✗ Hard to test

✗ Hard to scale

□ This is where **Dependency Injection (DI)** comes in.

□ **What is Dependency Injection?**

Dependency Injection means:

- Instead of creating objects inside a function,
- You **provide (inject) them from outside**

□ **Real Backend Example:**

Without DI:

```
def get_user():  
    db = Database() # tightly coupled ✗
```

With DI:

```
def get_user(db):  
    return db.get_user()
```

□ Benefits:

- Loose coupling
- Reusability
- Easier testing
- Cleaner architecture

□ In FastAPI, DI is implemented using:

□ **Depends()**

2. Core Concepts

2.1 What is Depends() in FastAPI?

Depends() is used to:

□ Inject dependencies into route functions

Basic Example:

```
from fastapi import Depends, FastAPI

app = FastAPI()

def get_message():
    return "Hello"

@app.get("/")
def home(msg: str = Depends(get_message)):
```

```
return {"message": msg}
```

□ **What Happens Internally?**

- FastAPI calls `get_message()`
- Injects result into `home()`

□ **Key Idea:**

- Function result becomes dependency

2.2 Why Dependency Injection is Important

Without DI:

```
def route():  
    db = Database()  
    user = db.get_user()
```

Problems:

- Hardcoded dependency
- Hard to test
- Not reusable

With DI:

```
def route(db):  
    return db.get_user()
```

□ Benefits:

- Clean code
- Easy testing
- Flexible design

2.3 Database Session Dependency (Very Important)

Database connection is one of the **most common dependencies**.

Example (Basic DB Session):

```
def get_db():  
    db = "DB Connection"  
    try:  
        yield db  
    finally:  
        print("Closing DB")
```

Using in Route:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
    return {"db": db}
```

□ Important Concept: **yield**

- **yield** → returns value
- After request → cleanup runs

□ Used for:

- Opening DB connection
- Closing DB connection

2.4 Dependency Lifecycle (Advanced)

Steps:

1. Dependency function runs
2. Value injected
3. Route executes
4. Cleanup runs (after **yield**)

Flow:

`get_db() → yield db → route → cleanup`

Backend Insight:

- Prevents resource leaks
- Ensures proper cleanup

2.5 Nested Dependencies

Dependencies can depend on other dependencies.

Example:

```
def get_db():  
    return "DB"  
  
def get_user(db = Depends(get_db)):  
    return f"user from {db}"  
  
@app.get("/")  
def home(user = Depends(get_user)):  
    return {"user": user}
```

□ Dependency chain:

- `home() → get_user() → get_db()`

2.6 Dependency for Authentication

Example:

```
from fastapi import HTTPException

def get_current_user(token: str):
    if token != "valid":
        raise HTTPException(status_code=401)
    return {"user": "Ali"}
```

Usage:

```
@app.get("/profile")
def profile(user = Depends(get_current_user)):
    return user
```

□ Used for:

- Authentication
- Authorization

2.7 Class-Based Dependencies

Example:

```
class Service:
    def get_data(self):
        return "data"

def get_service():
    return Service()

@app.get("/")
def home(service: Service = Depends(get_service)):
    return service.get_data()
```

□ Useful for:

- Service layer
- Business logic

2.8 Dependency Injection vs Global Variables

✗Bad (Global Variable):

```
db = Database()
```

✓ Good (DI):

```
def get_db():  
    return Database()
```

□ DI is:

- Safer
- Scalable
- Testable

2.9 Dependency Injection for Testing

Example:

```
def get_db():  
    return "real_db"
```

In Test:

```
def override_db():  
    return "mock_db"
```

```
app.dependency_overrides[get_db] = override_db
```

- Now tests use mock DB

Backend Benefit:

- No real DB calls
- Faster testing
- Controlled environment

3. Real-World Examples

Example 1: DB Dependency

```
def get_db():  
    return "DB"
```

Example 2: Service Layer

```
class UserService:  
    def get_user(self):  
        return {"name": "Ali"}
```

Example 3: Auth Dependency

```
def auth():  
    return {"user": "Ali"}
```

Example 4: Combined Dependency

```
def get_data(db = Depends(get_db)):  
    return f"data from {db}"
```

4. Code Examples (Important Patterns)

Pattern 1: Clean Route

```
@app.get("/users")  
def get_users(db = Depends(get_db)):  
    return db
```

Pattern 2: Service Injection

```
def get_service():  
    return Service()
```

Pattern 3: Auth Injection

```
def get_user():  
    return {"name": "Ali"}
```

Pattern 4: Nested Dependencies

```
def get_data(db = Depends(get_db)):  
    return db
```

5. Common Mistakes

✗ Creating DB inside route

✗ Not using `yield` for cleanup

✗ Overusing global variables

✗ Not understanding dependency chain

✗ Tight coupling

6. Interview Questions with Answers

Q1: What is dependency injection?

Providing dependencies from outside instead of creating inside

Q2: What is Depends()?

FastAPI tool for injecting dependencies

Q3: Why use DI?

- Clean code
- Reusability
- Testability

Q4: What is yield in dependency?

Used for setup + cleanup

Q5: What is DB session dependency?

Function that provides DB connection

Q6: What are nested dependencies?

Dependencies depending on other dependencies

Q7: Why DI is better than global variables?

More flexible and testable

Q8: How to override dependency in testing?

Using `dependency_overrides`

7. Summary (Quick Revision)

- Dependency Injection = provide dependencies externally
- `Depends()` → inject dependency
- Use `yield` for cleanup
- Common dependencies:
 - DB
 - Auth
 - Services

- Benefits:
 - Clean code
 - Reusable
 - Testable
- Avoid:
 - global variables
 - tight coupling

☐ Master dependency injection =

✓ Clean backend architecture

✓ Scalable systems

✓ Strong interview performance ☐

Chapter 3: Authentication & Security

(JWT, Password Hashing, OAuth Basics)

1. Introduction (Why This Matters in Backend)

Every real backend system must answer one critical question:

❑ **Who is the user and are they allowed to access this resource?**

This is handled by:

- **Authentication** → Who are you?
- **Authorization** → What can you do?

❑ **Real-World Examples:**

- Login to Instagram → authentication
- Access admin panel → authorization
- Online banking → strict security

❑ **What Happens Without Security?**

✗ Anyone can access private data

✗ Passwords can be stolen

✗ System can be hacked

❑ Backend developer responsibility:

- Secure user data
- Prevent unauthorized access
- Protect APIs

□ If you master this:

- You can build **secure backend systems**
- You pass **backend interviews easily**
- You understand real-world architecture

2. Core Concepts

2.1 Authentication vs Authorization

□ Authentication

Verifies identity:

□ "Are you really this user?"

Example:

- Username + password

□ Authorization

Checks permissions:

□ "What are you allowed to do?"

Example:

- Admin vs normal user

□ **Flow:**

Login → Authentication → Authorization → Access granted

2.2 Password Hashing (Very Important)

□ **Why Not Store Passwords Directly?**

✗ Storing plain passwords is dangerous

□ If database is hacked → all passwords exposed

□ **Solution: Hashing**

Hashing converts password into:

□ Irreversible string

Example:

```
password = "123456"  
hashed = "abc$xyz..."
```

- Cannot convert back to original password

2.3 Hashing with bcrypt

Install:

```
pip install passlib[bcrypt]
```

Example:

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"])

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(password: str, hashed: str):
    return pwd_context.verify(password, hashed)
```

□ Flow:

- User registers → password hashed
- User logs in → password verified

☐ Backend Rule:

✓ Never store plain password

2.4 JWT (JSON Web Token)

☐ **What is JWT?**

JWT is a **token used for authentication**.

☐ Instead of sending username/password every time:

☐ Server gives a token

☐ **Structure of JWT:**

HEADER.PAYLOAD.SIGNATURE

Example:

eyJhbGciOiJIc2Vy...eyJ1c2Vy...signature

☐ **How JWT Works:**

1. User logs in
2. Server verifies credentials
3. Server returns JWT token

4. Client stores token
5. Client sends token in every request

□ **Example Header:**

Authorization: Bearer <token>

2.5 JWT Implementation in FastAPI

Install:

```
pip install python-jose
```

Create Token:

```
from jose import jwt

SECRET_KEY = "secret"
ALGORITHM = "HS256"

def create_token(data: dict):
    return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)
```

Verify Token:

```
def verify_token(token: str):  
    return jwt.decode(token, SECRET_KEY,  
        algorithms=[ALGORITHM])
```

2.6 Protecting Routes (Authentication Dependency)

Example:

```
from fastapi import Depends, HTTPException  
  
def get_current_user(token: str):  
    try:  
        payload = verify_token(token)  
        return payload  
    except:  
        raise HTTPException(status_code=401)
```

Use in Route:

```
@app.get("/profile")  
def profile(user = Depends(get_current_user)):  
    return user
```

- ❑ Only authenticated users can access

2.7 OAuth Basics (High-Level)

□ What is OAuth?

OAuth allows:

- Login using third-party services

Examples:

- Login with Google
- Login with Facebook

□ Flow:

1. User clicks "Login with Google"
2. Redirect to Google
3. User logs in
4. Google returns token
5. Backend verifies token

□ Why Use OAuth?

- No need to store passwords
- More secure
- Better user experience

2.8 Authentication Flow (Complete)

Step-by-Step:

1. User registers → password hashed
2. User logs in → password verified
3. Server creates JWT
4. Client stores token
5. Client sends token in requests
6. Backend verifies token

2.9 Security Best Practices

- ☐ **Always Hash Passwords**
- ☐ **Use HTTPS**
- ☐ **Use JWT expiration**
- ☐ **Validate all inputs**
- ☐ **Avoid exposing sensitive data**

- ☐ **Example (Token Expiry):**

```
jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)
```

(Add expiry in real systems)

3. Real-World Examples

Example 1: Registration

```
def register(password: str):  
    return hash_password(password)
```

Example 2: Login

```
def login(password, hashed):  
    return verify_password(password, hashed)
```

Example 3: Token Creation

```
token = create_token({"user_id": 1})
```

Example 4: Protected Route

```
@app.get("/dashboard")  
def dashboard(user = Depends(get_current_user)):
```

```
return {"message": "Welcome"}
```

4. Code Examples (Important Patterns)

Pattern 1: Hash Password

```
hashed = hash_password("123")
```

Pattern 2: Verify Password

```
verify_password("123", hashed)
```

Pattern 3: Token-Based Auth

```
token = create_token({"user": "Ali"})
```

Pattern 4: Secure Endpoint

```
@app.get("/secure")  
def secure(user = Depends(get_current_user)):  
    return user
```

5. Common Mistakes

✗ Storing plain passwords

✗ Not verifying tokens

✗ Not using HTTPS

✗ Long-lived tokens without expiry

✗ Exposing sensitive data

6. Interview Questions with Answers

Q1: What is authentication?

Verifying user identity

Q2: What is authorization?

Checking permissions

Q3: What is JWT?

Token used for authentication

Q4: Why hash passwords?

To protect user data

Q5: What is bcrypt?

Password hashing algorithm

Q6: What is OAuth?

Third-party authentication system

Q7: How JWT works?

Login → token → verify

Q8: Where token is sent?

Authorization header

7. Summary (Quick Revision)

- Authentication → identity
 - Authorization → permissions
 - Always hash passwords
 - JWT → token-based authentication
 - OAuth → third-party login
 - Secure APIs using:
 - tokens
 - validation
 - HTTPS
- ☐ Master authentication =
- ✓ Secure backend systems
- ✓ Production-ready APIs
- ✓ Strong interview performance ☐

Chapter 4: Middleware & CORS

(Logging Middleware, Request/Response Flow, Rate Limiting, CORS Handling)

1. Introduction (Why This Matters in Backend)

In real backend systems, every request does not go directly to your route.

□ Instead, it passes through multiple layers:

- Logging
- Authentication
- Validation
- Rate limiting

These layers are handled by **middleware**.

□ **What is Middleware?**

Middleware is a function that:

□ Runs **before and after every request**

□ **Real Backend Flow:**

Client → Middleware → Route → Middleware → Response

❑ Why Middleware is Important?

- Logging requests
- Handling authentication globally
- Managing errors
- Adding headers
- Rate limiting

❑ What is CORS?

CORS = **Cross-Origin Resource Sharing**

- ❑ It controls which frontend can access your backend.

Example:

Frontend (localhost:3000) → Backend (localhost:8000)

- ❑ Different origins → CORS required
- ❑ If you don't understand middleware & CORS:
 - APIs may fail in frontend
 - Security issues may arise
- ❑ If you master this:
 - You can build **production-ready APIs**
 - You understand real backend flow

2. Core Concepts

2.1 Middleware in FastAPI

Basic Middleware Example:

```
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    print("Before request")

    response = await call_next(request)

    print("After request")
    return response
```

□ Explanation:

- `request` → incoming request
- `call_next()` → passes request to next layer
- Code before → runs before route
- Code after → runs after route

□ Flow:

Request → Middleware → Route → Middleware → Response

2.2 Request & Response Object

Access Request Data:

```
@app.middleware("http")
async def middleware(request: Request, call_next):
    print(request.url)
    response = await call_next(request)
    return response
```

Modify Response:

```
response.headers["X-Custom"] = "Value"
```

Backend Use:

- Add headers
- Track requests
- Modify responses

2.3 Logging Middleware (Very Important)

Example:

```
import time

@app.middleware("http")
async def logger(request: Request, call_next):
    start = time.time()

    response = await call_next(request)

    duration = time.time() - start
    print(f"{request.method} {request.url} - {duration}")

    return response
```

□ What It Does:

- Logs request method
- Logs URL
- Logs execution time

Backend Benefit:

- Monitor performance
- Debug issues
- Track usage

2.4 Error Handling Middleware

Example:

```
from fastapi.responses import JSONResponse

@app.middleware("http")
async def error_handler(request: Request, call_next):
    try:
        return await call_next(request)
    except Exception as e:
        return JSONResponse(
            status_code=500,
            content={"error": str(e)}
        )
```

Backend Benefit:

- Prevent crashes
- Return proper error response

2.5 Rate Limiting (Basic Concept)

What is Rate Limiting?

- Restrict number of requests per user

Example:

- Max 100 requests per minute

Simple Implementation:

```
calls = 0

@app.middleware("http")
async def limit(request: Request, call_next):
    global calls

    if calls > 5:
        return JSONResponse(
            status_code=429,
            content={"error": "Too many requests"}
        )

    calls += 1
    return await call_next(request)
```

Backend Insight:

- Prevent abuse
- Protect server

2.6 CORS (Cross-Origin Resource Sharing)

Problem:

Frontend cannot access backend due to:

- Different origin

Solution: Enable CORS

Example:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

□ Parameters:

Parameter	Meaning
allow_origins	Allowed domains
allow_methods	HTTP methods
allow_headers	Headers

2.7 Secure CORS Configuration (Important)

✗Bad:

```
allow_origins=["*"]
```

✓Good:

```
allow_origins=["http://localhost:3000"]
```

Backend Insight:

- Restrict origins in production
- Avoid security risks

2.8 Middleware Execution Order

Important Rule:

- Middleware runs in order of declaration

Example:

Middleware A → Middleware B → Route → B → A

□ Like a stack

3. Real-World Examples

Example 1: Logging System

```
@app.middleware("http")
async def logger(request: Request, call_next):
    print(request.url)
    return await call_next(request)
```

Example 2: API Security

```
if "Authorization" not in request.headers:
    return JSONResponse(status_code=401)
```

Example 3: Performance Monitoring

```
start = time.time()
```

Example 4: CORS Setup

```
allow_origins=["http://frontend.com"]
```

4. Code Examples (Important Patterns)

Pattern 1: Logging Middleware

```
print(request.method, request.url)
```

Pattern 2: Add Header

```
response.headers["X-App"] = "FastAPI"
```

Pattern 3: Block Request

```
return JSONResponse(status_code=403)
```

Pattern 4: Timing Middleware

```
duration = time.time() - start
```

5. Common Mistakes

✗ Not using middleware for common logic

✗ Logging inside every route

✗ Using open CORS in production

✗ Not handling errors globally

✗ Poor rate limiting implementation

6. Interview Questions with Answers

Q1: What is middleware?

Function that runs before and after request

Q2: Why middleware is used?

- Logging
- Auth
- Validation

Q3: What is CORS?

Cross-origin resource sharing

Q4: Why CORS is needed?

Allow frontend to access backend

Q5: What is rate limiting?

Restrict number of requests

Q6: What is call_next?

Pass request to next layer

Q7: Can middleware modify response?

Yes

Q8: Middleware execution order?

Top → bottom → reverse

7. Summary (Quick Revision)

- Middleware runs before & after request
- Used for:
 - Logging
 - Auth
 - Error handling
- CORS controls access
- Rate limiting protects server
- Always:
 - restrict CORS
 - log requests
 - handle errors

□ Master middleware =

✓ Production-ready backend

✓ Secure APIs

✓ Better system control ☐

Chapter 5: API Design Best Practices

(REST Principles, Status Codes, Pagination, Clean API Design)

1. Introduction (Why This Matters in Backend)

Building an API is easy.

Designing a **good API** is hard.

□ What is a Good API?

A good API is:

- Easy to understand
- Consistent
- Scalable
- Predictable

□ Real Backend Problem

Bad API design leads to:

- ✗ Confusion for frontend developers
- ✗ Difficult maintenance
- ✗ Bugs and inconsistencies

□ Example:

Bad API:

/getUserData
/createNewUserNow

Good API:

GET /users/{id}
POST /users

□ Interviewers expect you to know:

- REST principles
- Proper status codes
- API structure

□ If you master this:

- You build **professional APIs**
- You stand out in interviews
- You write **production-ready systems**

2. Core Concepts

2.1 REST Principles (Very Important)

REST = **Representational State Transfer**

It is the standard way to design APIs.

□ Key Principles:

1. Resource-Based Design

APIs should represent **resources (nouns)**, not actions.

✗Bad:

/getUser

✓Good:

/users

2. Use HTTP Methods Properly

Method	Purpose
GET	Retrieve data
POST	Create resource
PUT	Update resource
DELETE	Delete resource

3. Statelessness

Each request should contain all required data.

- ❑ Server should not remember previous requests

4. Uniform Interface

Use consistent structure across APIs.

Example:

GET /users

GET /users/{id}

POST /users

DELETE /users/{id}

Backend Insight:

- Consistency is more important than creativity

2.2 Naming Conventions (Clean API Design)

- ❑ Use Nouns (Not Verbs)

✗ /getUsers

✓ /users

☐ **Use Plural Names**

✗ /user

✓ /users

☐ **Nested Resources**

/users/1/orders

☐ **Use Hyphens (Readable URLs)**

/user-profile

☐ **Backend Insight:**

- API should be readable like English

2.3 Status Codes (Deep Understanding)

☐ **Why Status Codes Matter?**

- Communicate result of request

- Help frontend handle responses

□ **Common Status Codes:**

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error

□ **Example:**

```
from fastapi import HTTPException

@app.get("/users/{id}")
def get_user(id: int):
    if id < 0:
        raise HTTPException(status_code=400, detail="Invalid ID")
```

□ **Backend Insight:**

- Always return correct status codes
- Never return 200 for errors

2.4 Request & Response Structure

□ Standard Response Format

```
{  
  "status": "success",  
  "data": {...},  
  "message": "User fetched"  
}
```

□ Error Response Format

```
{  
  "status": "error",  
  "message": "User not found"  
}
```

□ Why Standard Format?

- Consistency
- Easy frontend integration
- Easier debugging

2.5 Pagination (Very Important)

□ Problem:

Large datasets:

- 10,000 users
- Cannot return all at once

□ Solution: Pagination

Example:

```
@app.get("/users")
def get_users(limit: int = 10, offset: int = 0):
    return {"limit": limit, "offset": offset}
```

□ Query Example:

```
/users?limit=10&offset=20
```

□ Types of Pagination:

1. Offset-based

2. Cursor-based (advanced)

□ Backend Insight:

- Always paginate large data
- Improves performance

2.6 Filtering & Sorting

□ Filtering:

/users?age=25

□ Sorting:

/users?sort=asc

Example:

```
@app.get("/users")
def get_users(age: int = None, sort: str = "asc"):
    return {"age": age, "sort": sort}
```

Backend Insight:

- Improves API usability

2.7 Versioning APIs

□ Why Versioning?

APIs change over time.

Example:

/v1/users

/v2/users

Backend Insight:

- Prevents breaking existing clients

2.8 Idempotency (Interview Favorite)

□ What is Idempotent?

Same request → same result

Examples:

Method	Idempotent
GET	Yes
PUT	Yes
DELETE	Yes
POST	No

Backend Insight:

- Important for system reliability

3. Real-World Examples

Example 1: RESTful User API

```
@app.get("/users")
def get_users():
    return []
```

Example 2: Create Resource

```
@app.post("/users")
def create_user():
    return {"status": "created"}
```

Example 3: Pagination

```
@app.get("/users")
def get_users(limit: int = 10):
    return {"limit": limit}
```

Example 4: Error Handling

```
raise HTTPException(status_code=404, detail="Not found")
```

4. Code Examples (Important Patterns)

Pattern 1: Clean URL

```
/users/{id}
```

Pattern 2: Query Parameters

```
/users?limit=10
```

Pattern 3: Standard Response

```
return {"status": "success", "data": data}
```

Pattern 4: Error Response

```
raise HTTPException(status_code=400)
```

5. Common Mistakes

✗ Using verbs in URLs

✗ Not using status codes properly

✗ Returning too much data

✗ No pagination

✗ Inconsistent response format

✗ Not versioning APIs

6. Interview Questions with Answers

Q1: What is REST?

Standard for designing APIs

Q2: Why use HTTP methods?

Define action on resource

Q3: What is pagination?

Limit data returned

Q4: What is idempotency?

Same request → same result

Q5: Why version APIs?

Avoid breaking clients

Q6: What is good API design?

- Consistent
- Simple
- Predictable

Q7: Why use status codes?

Communicate response result

Q8: What is resource-based design?

Using nouns instead of verbs

7. Summary (Quick Revision)

- REST → standard API design
- Use:
 - nouns
 - HTTP methods
- Always:
 - use status codes

- paginate data
 - keep consistent format
- Avoid:
 - bad naming
 - large responses
 - inconsistent APIs

☐ Master API design =

✓ Professional backend systems

✓ Better frontend integration

✓ Strong interview performance ☐

Chapter 6: Flask Essentials

(App Factory, Blueprints, Project Structure, When to Use Flask)

1. Introduction (Why This Matters in Backend)

So far, you learned FastAPI — modern and async.

But in real industry, you will also encounter **Flask**.

□ Flask is:

- Lightweight
- Flexible
- Widely used in legacy + production systems

□ **Why Learn Flask?**

- Many companies still use Flask
- Great for understanding backend fundamentals
- Gives you control over architecture

□ **FastAPI vs Flask**

Feature	FastAPI	Flask
Performance	High (async)	Medium (sync)
Validation	Built-in	Manual
Simplicity	Medium	Very simple
Use Case	Modern APIs	Flexible apps

□ Interview Insight:

- Knowing both gives you an advantage

□ If you master Flask:

- You understand backend fundamentals deeply
- You can work on existing production systems

2. Core Concepts

2.1 What is Flask?

Flask is a **micro web framework**.

□ Micro means:

- Minimal features
- You add what you need

Basic Example:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Flask"
```

Running Flask App:

```
flask run
```

2.2 Routing in Flask

Example:

```
@app.route("/users")
def get_users():
    return "Users"
```

Methods:

```
@app.route("/users", methods=["POST"])
def create_user():
    return "Created"
```

2.3 Request & Response Handling

Access Request Data:

```
from flask import request

@app.route("/data", methods=["POST"])
def data():
    return request.json
```

Return JSON Response:

```
from flask import jsonify

@app.route("/")
def home():
    return jsonify({"message": "Hello"})
```

2.4 Application Structure (Problem)

✗Bad (Single File)

```
# everything in one file
```

Problems:

- Hard to scale
- Hard to maintain

□ Solution:

App Factory Pattern

2.5 App Factory Pattern (Very Important)

□ **What is App Factory?**

Instead of creating app globally:

□ Create app inside a function

Example:

```
from flask import Flask

def create_app():
    app = Flask(__name__)

    @app.route("/")
    def home():
        return "Hello"

    return app
```

□ **Why Use It?**

- Better structure

- Easier testing
- Supports multiple configurations

Backend Insight:

- Used in production-level Flask apps

2.6 Blueprints (Very Important)

□ What is Blueprint?

Blueprint = **modular component of app**

- Splits large app into smaller parts

Example:

```
from flask import Blueprint

user_bp = Blueprint("user", __name__)

@user_bp.route("/users")
def get_users():
    return "Users"
```

Register Blueprint:

```
app.register_blueprint(user_bp)
```

□ Why Blueprints?

- Organize code
- Separate modules
- Scalable architecture

Example Structure:

```
app/  
├── user/  
│   ├── routes.py  
│   └── models.py
```

2.7 Flask Project Structure (Best Practice)

Recommended Structure:

```
project/  
├── app/  
│   └── __init__.py
```

```
|
|   ├── routes.py
|   ├── models.py
|   └── services.py
|
├── config.py
└── run.py
```

□ **Explanation:**

- `routes.py` → endpoints
- `models.py` → database models
- `services.py` → business logic

□ Separation of concerns

2.8 Flask Extensions

Flask is minimal, so we use extensions:

Common Extensions:

- Flask-SQLAlchemy → database
- Flask-JWT → authentication
- Flask-Migrate → migrations

Example:

```
from flask_sqlalchemy import SQLAlchemy
```

```
db = SQLAlchemy()
```

2.9 Flask vs FastAPI (When to Use)

Use Flask When:

- Small projects
- Full control needed
- Legacy systems

Use FastAPI When:

- High-performance APIs
- Async required
- Modern systems

3. Real-World Examples

Example 1: Simple API

```
@app.route("/users")
def users():
    return "Users"
```

Example 2: JSON API

```
@app.route("/data")
def data():
    return jsonify({"key": "value"})
```

Example 3: Blueprint Usage

```
user_bp = Blueprint("user", __name__)
```

Example 4: App Factory

```
app = create_app()
```

4. Code Examples (Important Patterns)

Pattern 1: Modular Structure

```
app.register_blueprint(user_bp)
```

Pattern 2: Clean Route

```
@app.route("/")
def home():
    return "Home"
```

Pattern 3: JSON Response

```
return jsonify({"status": "ok"})
```

Pattern 4: App Factory

```
def create_app():
    return Flask(__name__)
```

5. Common Mistakes

✗ Writing everything in one file

✗ Not using blueprints

✗ Hardcoding configurations

✗ Mixing business logic in routes

✗ Not using app factory

6. Interview Questions with Answers

Q1: What is Flask?

Lightweight Python web framework

Q2: What is app factory?

Function that creates app instance

Q3: What is blueprint?

Modular part of Flask app

Q4: Why Flask is called micro?

Minimal features

Q5: Flask vs FastAPI?

- Flask → flexible
- FastAPI → fast + modern

Q6: Why use blueprints?

Organize large apps

Q7: What are Flask extensions?

Libraries to add features

Q8: When to use Flask?

Small or legacy systems

7. Summary (Quick Revision)

- Flask = lightweight backend framework

- Routing using `@app.route()`
- Use:
 - App factory
 - Blueprints
- Follow:
 - clean structure
 - separation of concerns
- Use extensions for features

☐ Master Flask =

- ✓ Strong backend fundamentals
- ✓ Ability to work on real systems
- ✓ Better interview performance ☐

□ **PART 2: Databases**

Chapter 7: SQL Fundamentals

(SELECT, JOIN, GROUP BY, Subqueries)

1. Introduction (Why This Matters in Backend)

Backend applications are nothing without **data**.

Where is data stored?

□ In databases

□ **Real Backend Examples:**

- User login → fetch user from DB
- E-commerce → store products
- Banking → store transactions

□ As a backend developer, you must:

- Retrieve data
- Insert data
- Update data
- Delete data

□ This is done using **SQL (Structured Query Language)**

□ **Why SQL is Important?**

- Used in almost every backend system
- Required in interviews
- Critical for performance

□ If you master SQL:

- You can build real applications
- You can optimize queries
- You will clear backend interviews

2. Core Concepts

2.1 What is a Database?

A database is a system to **store and manage data**.

□ **Types of Databases:**

- Relational (SQL) → MySQL, PostgreSQL
- NoSQL → MongoDB

□ Table Structure:

Users Table

id	name	age
1	Ali	25
2	Sara	30

□ Data stored in rows and columns

2.2 SELECT (Most Important)

SELECT is used to **retrieve data**.

Basic Syntax:

```
SELECT * FROM users;
```

Specific Columns:

```
SELECT name, age FROM users;
```

With Condition:

```
SELECT * FROM users WHERE age > 25;
```

Sorting:

```
SELECT * FROM users ORDER BY age DESC;
```

Limit:

```
SELECT * FROM users LIMIT 5;
```

Backend Insight:

- Most used query in backend

2.3 WHERE Clause (Filtering Data)

Example:

```
SELECT * FROM users WHERE name = 'Ali';
```

Operators:

Operator	Meaning
=	Equal
>	Greater
<	Less
AND	Both conditions
OR	Either condition

Example:

```
SELECT * FROM users WHERE age > 20 AND age < 30;
```

2.4 JOIN (Very Important)

JOIN is used to combine data from multiple tables.

Example Tables:

Users

id		name
1		Ali

Orders

id		user_id		product
1		1		Phone

INNER JOIN:

```
SELECT users.name, orders.product
FROM users
JOIN orders ON users.id = orders.user_id;
```

Types of JOIN:

Type	Meaning
INNER JOIN	Matching rows
LEFT JOIN	All left + matches
RIGHT JOIN	All right + matches

Backend Insight:

- Used in real applications frequently

2.5 GROUP BY (Aggregation)

Used to group data.

Example:

```
SELECT age, COUNT(*)
FROM users
GROUP BY age;
```

Common Functions:

Function	Purpose
COUNT()	Count rows
SUM()	Total
AVG()	Average
MAX()	Maximum
MIN()	Minimum

Example:

```
SELECT AVG(age) FROM users;
```

2.6 HAVING Clause

Used with GROUP BY.

Example:

```
SELECT age, COUNT(*)  
FROM users  
GROUP BY age  
HAVING COUNT(*) > 1;
```

2.7 Subqueries (Advanced Concept)

A query inside another query.

Example:

```
SELECT * FROM users  
WHERE age > (SELECT AVG(age) FROM users);
```

Backend Use:

- Complex filtering
- Nested logic

2.8 INSERT, UPDATE, DELETE

INSERT:

```
INSERT INTO users (name, age)  
VALUES ('Ali', 25);
```

UPDATE:

```
UPDATE users  
SET age = 26  
WHERE name = 'Ali';
```

DELETE:

```
DELETE FROM users  
WHERE name = 'Ali';
```

2.9 Indexes (Performance Concept)

What is Index?

- Improves query speed
- Works like book index

Example:

```
CREATE INDEX idx_name ON users(name);
```

Backend Insight:

- Important for large data

3. Real-World Examples

Example 1: Fetch Users

```
SELECT * FROM users;
```

Example 2: User Orders

```
SELECT users.name, orders.product  
FROM users  
JOIN orders ON users.id = orders.user_id;
```

Example 3: Count Users

```
SELECT COUNT(*) FROM users;
```

Example 4: Filter Data

```
SELECT * FROM users WHERE age > 25;
```

4. Code Examples (Backend Pattern)

Pattern 1: Fetch Data

```
SELECT * FROM table;
```

Pattern 2: Filter Data

```
SELECT * FROM table WHERE condition;
```

Pattern 3: Join Tables

```
SELECT * FROM a JOIN b;
```

Pattern 4: Aggregate

```
SELECT COUNT(*) FROM table;
```

5. Common Mistakes

✗ Using SELECT * in production

✗ Missing indexes

✗ Incorrect JOIN conditions

✗ Not using WHERE clause

✗ Ignoring performance

6. Interview Questions with Answers

Q1: What is SQL?

Language to interact with database

Q2: What is SELECT?

Retrieve data

Q3: What is JOIN?

Combine tables

Q4: What is GROUP BY?

Group data

Q5: Difference between WHERE and HAVING?

- WHERE → filter rows
- HAVING → filter groups

Q6: What is subquery?

Query inside query

Q7: What is index?

Improves query speed

Q8: What is primary key?

Unique identifier

7. Summary (Quick Revision)

- SQL = database query language
 - Key operations:
 - SELECT
 - INSERT
 - UPDATE
 - DELETE
 - JOIN → combine tables
 - GROUP BY → aggregation
 - Subquery → nested query
 - Index → performance boost
- ☐ Master SQL fundamentals =
- ✓ Strong backend skills
 - ✓ Efficient data handling
 - ✓ High interview success ☐

Chapter 8: Advanced SQL

(Window Functions, CTEs, Indexing, Query Optimization)

1. Introduction (Why This Matters in Backend)

Basic SQL is enough to **build small applications**, but not enough for:

- Large-scale systems
- High-performance APIs
- Complex analytics

□ Real Backend Problems:

- Slow queries
- Duplicate data processing
- Complex joins
- Large datasets (millions of rows)

□ This is where **Advanced SQL** comes in.

□ What You Will Learn:

- Window functions → advanced calculations
- CTEs → clean query structure
- Indexing → faster queries
- Query optimization → performance improvement

□ If you master this:

- You write **efficient database queries**
- You handle **large-scale systems**
- You crack **backend interviews easily**

2. Core Concepts

2.1 Window Functions (Very Important)

□ **What are Window Functions?**

Window functions perform calculations:

- Across a set of rows
- Without grouping them

□ **Difference from GROUP BY:**

Feature	GROUP BY	Window Function
Output rows	Reduced	Same as input
Use case	Aggregation	Row-wise analysis

□ **Example Table:**

Users

id | name | age

1		Ali		25
2		Sara		30
3		John		25

❑ **ROW_NUMBER()**

Assigns row numbers.

```
SELECT name,  
ROW_NUMBER() OVER() AS row_num  
FROM users;
```

❑ **PARTITION BY**

Group rows without collapsing.

```
SELECT name, age,  
COUNT(*) OVER(PARTITION BY age) AS count_age  
FROM users;
```

❑ **RANK()**

```
SELECT name, age,  
RANK() OVER(ORDER BY age DESC) AS rank
```

```
FROM users;
```

□ Backend Use:

- Ranking users
- Analytics
- Reports

2.2 Common Window Functions

Function	Use
ROW_NUMBER()	Unique row number
RANK()	Ranking with gaps
DENSE_RANK()	Ranking without gaps
LAG()	Previous row
LEAD()	Next row

Example: LAG()

```
SELECT name, age,  
LAG(age) OVER() AS prev_age  
FROM users;
```

2.3 CTE (Common Table Expression)

□ What is CTE?

CTE = temporary result set used in query

Syntax:

```
WITH temp AS (  
    SELECT * FROM users  
)  
SELECT * FROM temp;
```

□ Why Use CTE?

- Improves readability
- Breaks complex queries
- Reusable logic

Example:

```
WITH avg_age AS (  
    SELECT AVG(age) AS avg_val FROM users  
)  
SELECT * FROM users
```

```
WHERE age > (SELECT avg_val FROM avg_age);
```

Backend Insight:

- Used in complex reports
- Improves maintainability

2.4 Indexing (Performance Optimization)

☐ What is Index?

Index is a **data structure** that:

- ☐ Improves query speed

☐ Without Index:

- Full table scan
- Slow

☐ With Index:

- Fast lookup

Example:

```
CREATE INDEX idx_name ON users(name);
```

☐ **Types of Indexes:**

Type	Use
Primary	Unique key
Unique	No duplicates
Composite	Multiple columns

☐ **Backend Insight:**

- Index improves read speed
- Too many indexes slow write operations

2.5 Query Optimization (Very Important)

☐ **Problem:**

```
SELECT * FROM users;
```

☐ Fetches all data → slow

☐ **Optimized:**

```
SELECT name FROM users;
```

☐ **Best Practices:**

1. Avoid SELECT *

2. Use WHERE clause

```
SELECT * FROM users WHERE age > 25;
```

3. Use Indexes

4. Limit Data

```
SELECT * FROM users LIMIT 10;
```

5. Avoid Nested Queries (Sometimes)

Use JOIN instead

2.6 Execution Plan (Advanced)

☐ What is Execution Plan?

Shows how query runs internally

Example:

```
EXPLAIN SELECT * FROM users;
```

Backend Insight:

- Helps identify slow queries
- Used in optimization

2.7 Joins Optimization

❑ **Bad Join:**

```
SELECT * FROM a, b;
```

❑ **Good Join:**

```
SELECT * FROM a  
JOIN b ON a.id = b.id;
```

Backend Insight:

- Always use proper join conditions

3. Real-World Examples

Example 1: Ranking Users

```
SELECT name,  
RANK() OVER(ORDER BY age DESC)  
FROM users;
```

Example 2: Average Filter

```
WITH avg_age AS (  
    SELECT AVG(age) AS avg_val FROM users  
)  
SELECT * FROM users WHERE age > avg_val;
```

Example 3: Performance Query

```
SELECT name FROM users WHERE age > 25;
```

Example 4: Index Usage

```
CREATE INDEX idx_age ON users(age);
```

4. Code Examples (Backend Patterns)

Pattern 1: Window Function

```
ROW_NUMBER() OVER()
```

Pattern 2: CTE

```
WITH temp AS (...)
```

Pattern 3: Index

```
CREATE INDEX idx ON table(column);
```

Pattern 4: Optimization

```
SELECT column FROM table;
```

5. Common Mistakes

✗ Using SELECT *

✗ Not using indexes

✗ Overusing subqueries

✗ Poor JOIN conditions

✗ Ignoring execution plan

6. Interview Questions with Answers

Q1: What is window function?

Performs calculation across rows

Q2: What is CTE?

Temporary query result

Q3: What is index?

Improves query performance

Q4: Difference between RANK and DENSE_RANK?

- RANK → gaps
- DENSE_RANK → no gaps

Q5: Why avoid SELECT *?

Slow and unnecessary data

Q6: What is execution plan?

Shows query execution

Q7: What is query optimization?

Improving query performance

Q8: When to use index?

For frequently searched columns

7. Summary (Quick Revision)

- Window functions → advanced calculations
- CTE → clean queries
- Index → faster queries
- Optimize queries:
 - avoid SELECT *
 - use WHERE
 - use indexes

- Execution plan → analyze performance

☐ Master advanced SQL =

✓ High-performance backend

✓ Efficient database queries

✓ Strong interview advantage ☐

Chapter 9: Database Design

(Normalization, Relationships, Schema Design, Real-World Modeling)

1. Introduction (Why This Matters in Backend)

Writing queries is one part of backend development.

Designing the **database correctly** is even more important.

□ Why Database Design Matters?

A poorly designed database leads to:

- ✗ Duplicate data
- ✗ Inconsistent data
- ✗ Difficult queries
- ✗ Poor performance

□ Real Example:

Bad design:

Users Table

name		order1		order2		order3
------	--	--------	--	--------	--	--------

Good design:

Users Table + Orders Table

☐ Backend developers must:

- Design tables
- Define relationships
- Ensure data consistency

☐ If you master this:

- You build scalable systems
- You write efficient queries
- You perform well in interviews

2. Core Concepts

2.1 What is Database Design?

Database design is:

- ☐ Organizing data into tables
- ☐ Defining relationships between tables

☐ **Goals:**

- Reduce redundancy
- Improve performance

- Maintain consistency

2.2 Normalization (Very Important)

Normalization means:

- ☐ Organizing data to remove duplication

- ☐ **Why Normalize?**

- Avoid repeated data
- Reduce storage
- Improve consistency

2.3 Normal Forms (Interview Important)

- ☐ **1st Normal Form (1NF)**

Rule:

- No repeating groups
- Atomic values

✗Bad:

User

name	phones
Ali	123,456

✓ Good:

UserPhones

name	phone
Ali	123
Ali	456

□ 2nd Normal Form (2NF)

Rule:

- Remove partial dependency

□ Data should depend on **whole primary key**

□ 3rd Normal Form (3NF)

Rule:

- Remove transitive dependency

✗ Bad:

User

id | city_id | city_name

✓ Good:

Users Table

id | city_id

City Table

id | city_name

Backend Insight:

- Most systems use up to 3NF

2.4 Relationships (Very Important)

□ Types of Relationships:

1. One-to-One

User → Profile

2. One-to-Many (Most Common)

User → Orders

Example:

Users

id | name

Orders

id | user_id

3. Many-to-Many

Students ↔ Courses

Example:

Student_Course

student_id | course_id

Backend Insight:

- Most real systems use:
 - One-to-many
 - Many-to-many

2.5 Primary Key & Foreign Key

□ Primary Key

- Unique identifier
- Cannot be null

```
id INT PRIMARY KEY
```

□ Foreign Key

- Links tables

```
user_id INT REFERENCES users(id)
```

Backend Insight:

- Ensures data integrity

2.6 Schema Design (Practical)

Example: E-commerce System

Users Table:

id | name | email

Products Table:

id | name | price

Orders Table:

id | user_id | product_id

Relationships:

- User → Orders
- Product → Orders

2.7 Denormalization (Advanced Concept)

□ What is Denormalization?

- Adding redundancy for performance

Example:

Orders Table

user_name stored directly

Why Use?

- Faster queries
- Less joins

Trade-off:

Advantage	Disadvantage
Fast	Redundant data

2.8 Constraints (Data Integrity)

Types:

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY

Example:

email VARCHAR UNIQUE NOT NULL

Backend Insight:

- Prevent invalid data

2.9 ER Diagram (Conceptual Understanding)

□ What is ER Diagram?

Shows:

- Entities (tables)
- Relationships

Example:

User → Order → Product

Backend Insight:

- Used before building DB

3. Real-World Examples

Example 1: User-Order Relationship

User → Orders (One-to-Many)

Example 2: Many-to-Many

Students ↔ Courses

Example 3: Normalized Data

Separate tables for:

- User
- Address

Example 4: Denormalized Data

Store extra fields for speed

4. Code Examples (SQL Design)

Create Table:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR  
);
```

Foreign Key:

```
CREATE TABLE orders (  
    id INT,  
    user_id INT REFERENCES users(id)
```

);

Many-to-Many Table:

```
CREATE TABLE student_course (  
    student_id INT,  
    course_id INT  
);
```

Constraints:

```
email VARCHAR UNIQUE NOT NULL
```

5. Common Mistakes

✗ Not normalizing data

✗ Too many tables (over-normalization)

✗ Missing foreign keys

✗ Poor relationships

✗ No constraints

6. Interview Questions with Answers

Q1: What is normalization?

Removing duplicate data

Q2: What is 1NF?

Atomic values

Q3: What is 2NF?

No partial dependency

Q4: What is 3NF?

No transitive dependency

Q5: What is primary key?

Unique identifier

Q6: What is foreign key?

Links tables

Q7: One-to-many example?

User → Orders

Q8: What is denormalization?

Adding redundancy for performance

7. Summary (Quick Revision)

- Database design = organizing data
- Normalize data:
 - 1NF
 - 2NF
 - 3NF

- Relationships:
 - One-to-one
 - One-to-many
 - Many-to-many
- Keys:
 - Primary
 - Foreign
- Use constraints for integrity

☐ Master database design =

✓ Scalable systems

✓ Clean data structure

✓ Strong interview performance ☐

Chapter 10: Transactions & ACID

(Transactions, ACID Properties, Isolation Levels, Deadlocks)

1. Introduction (Why This Matters in Backend)

In backend systems, data operations are not always simple.

☐ Real-World Scenario:

User transfers money:

1. Deduct from Account A
2. Add to Account B

Problem:

If system crashes after step 1:

✗ Money lost

✗ Data inconsistency

☐ Solution:

☐ Transactions

□ What is a Transaction?

A transaction is a **group of operations executed as a single unit**.

□ Either:

- All succeed ✓
- Or all fail ✗

□ Transactions are critical for:

- Banking systems
- E-commerce orders
- Inventory updates

□ If you master this:

- You build **reliable backend systems**
- You avoid **data corruption**
- You answer **advanced interview questions**

2. Core Concepts

2.1 What is a Transaction?

Example:

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;
```

□ **Key Commands:**

Command	Meaning
BEGIN	Start transaction
COMMIT	Save changes
ROLLBACK	Undo changes

Example with Error:

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

-- error happens here

ROLLBACK;
```

□ No data is changed

2.2 ACID Properties (Very Important)

ACID ensures **reliable transactions**.

□ **A = Atomicity**

- All operations succeed or none

Example:

- Transfer money → both debit & credit must succeed

□ **C = Consistency**

- Database remains valid

Example:

- Balance cannot be negative

□ **I = Isolation**

- Transactions don't interfere

Example:

- Two users updating same data

□ **D = Durability**

- Once committed → data is permanent

Example:

- Data saved even after crash

Backend Insight:

- ACID = foundation of reliable systems

2.3 Isolation Levels (Interview Favorite)

Isolation defines how transactions interact.

□ **Types:**

Level	Problem Prevented
Read Uncommitted	None
Read Committed	Dirty read
Repeatable Read	Non-repeatable read
Serializable	All issues

□ **Common Problems:**

1. Dirty Read

Reading uncommitted data

2. Non-repeatable Read

Same query → different result

3. Phantom Read

New rows appear during query

□ Example:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Backend Insight:

- Most systems use:
 - Read Committed
 - Repeatable Read

2.4 Deadlocks (Very Important)

□ What is Deadlock?

Two transactions waiting for each other:

Example:

Transaction A → locks row 1

Transaction B → locks row 2

A waits for row 2

B waits for row 1

□ Deadlock occurs

□ Result:

- System stuck
- DB kills one transaction

2.5 Deadlock Prevention

□ Techniques:

1. Access resources in same order

2. Keep transactions short

3. Avoid unnecessary locks

Backend Insight:

- Deadlocks are common in high concurrency systems

2.6 Transaction in Backend Code

Example (Pseudo Code):

```
try:
    begin_transaction()

    debit()
    credit()

    commit()
except:
    rollback()
```

Backend Framework Example:

```
session.begin()

try:
    session.commit()
except:
    session.rollback()
```

2.7 Locking Mechanism

□ What is Lock?

Lock prevents multiple transactions from modifying same data.

Types:

Lock Type	Use
Shared	Read
Exclusive	Write

Backend Insight:

- Locks ensure consistency
- Too many locks reduce performance

2.8 Real-World Transaction Example

E-commerce Order:

1. Check stock
2. Deduct stock
3. Create order

☐ All steps must succeed

Example:

```
BEGIN;
```

```
UPDATE products SET stock = stock - 1 WHERE id = 1;
```

```
INSERT INTO orders VALUES (...);
```

```
COMMIT;
```

3. Real-World Examples

Example 1: Bank Transfer

```
UPDATE accounts SET balance = balance - 100;  
UPDATE accounts SET balance = balance + 100;
```

Example 2: Rollback Scenario

```
ROLLBACK;
```

Example 3: Isolation Level

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Example 4: Deadlock Scenario

Two transactions waiting for each other

4. Code Examples (Backend Patterns)

Pattern 1: Transaction Block

```
BEGIN;  
...  
COMMIT;
```

Pattern 2: Rollback

```
ROLLBACK;
```

Pattern 3: Isolation Level

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Pattern 4: Safe Operation

```
try:  
    commit()  
except:  
    rollback()
```

5. Common Mistakes

✗ Not using transactions

✗ Long-running transactions

✗ Ignoring isolation levels

✗ Not handling rollback

✗ Causing deadlocks

6. Interview Questions with Answers

Q1: What is a transaction?

Group of operations executed together

Q2: What is ACID?

- Atomicity
- Consistency
- Isolation

- Durability

Q3: What is atomicity?

All or nothing execution

Q4: What is isolation?

Transactions don't interfere

Q5: What is deadlock?

Two transactions waiting for each other

Q6: How to prevent deadlock?

- Same order
- Short transactions

Q7: What is rollback?

Undo changes

Q8: What is isolation level?

Defines transaction behavior

7. Summary (Quick Revision)

- Transaction = group of operations
 - ACID ensures reliability
 - Use:
 - BEGIN
 - COMMIT
 - ROLLBACK
 - Isolation levels control concurrency
 - Deadlocks occur in conflicts
-
- ☐ Master transactions =
 - ✓ Reliable backend systems
 - ✓ Safe data handling
 - ✓ Strong interview performance ☐

Chapter 11: SQLAlchemy ORM

(Models, Relationships, Sessions, Query Optimization)

1. Introduction (Why This Matters in Backend)

So far, you learned SQL queries.

But in real backend applications, we rarely write raw SQL everywhere.

- Instead, we use **ORM (Object Relational Mapping)**

- **What is ORM?**

ORM allows you to:

- Interact with database using Python code instead of SQL

Example:

SQL:

```
SELECT * FROM users;
```

ORM:

```
session.query(User).all()
```

- Benefits:

- Cleaner code
- Less SQL writing
- Easier maintenance

□ What is SQLAlchemy?

SQLAlchemy is the **most popular ORM in Python**.

Used in:

- FastAPI
- Flask
- Production systems

□ If you master SQLAlchemy:

- You can build real backend systems
- You understand database abstraction
- You perform well in interviews

2. Core Concepts

2.1 ORM vs Raw SQL

Raw SQL:

- More control
- More complex

ORM:

- Easier to use
- Pythonic

□ Best practice:

- Use ORM for most cases
- Use raw SQL for complex queries

2.2 Creating Models (Very Important)

□ What is a Model?

Model = Python class representing a database table

Example:

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
```

```
name = Column(String)
```

□ **Explanation:**

- `__tablename__` → table name
- `Column` → table column
- `primary_key=True` → unique id

2.3 Creating Database & Tables

```
from sqlalchemy import create_engine  
  
engine = create_engine("sqlite:///test.db")  
  
Base.metadata.create_all(engine)
```

- Creates database tables

2.4 Sessions (Very Important)

□ **What is Session?**

Session = connection between app and database

Example:

```
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(bind=engine)
session = SessionLocal()
```

□ Why Session?

- Execute queries
- Manage transactions

2.5 CRUD Operations (Core Backend Work)

□ Create (Insert)

```
user = User(name="Ali")
session.add(user)
session.commit()
```

□ **Read (Select)**

```
users = session.query(User).all()
```

□ **Filter**

```
user = session.query(User).filter(User.name ==  
"Ali").first()
```

□ **Update**

```
user.name = "Sara"  
session.commit()
```

□ **Delete**

```
session.delete(user)  
session.commit()
```

2.6 Relationships (Very Important)

□ One-to-Many Example

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    orders = relationship("Order", back_populates="user")

class Order(Base):
    __tablename__ = "orders"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    user = relationship("User", back_populates="orders")
```

□ Key Concepts:

- ForeignKey → links tables
- relationship() → defines relation

Backend Insight:

- Used in almost every real app

2.7 Lazy vs Eager Loading

□ Lazy Loading

- Loads data when accessed

□ Eager Loading

- Loads data immediately

Example:

```
from sqlalchemy.orm import joinedload  
  
session.query(User).options(joinedload(User.orders))
```

Backend Insight:

- Avoid N+1 query problem

2.8 Query Optimization (Important)

❑ **Bad Query:**

```
session.query(User).all()
```

❑ **Better:**

```
session.query(User.name).all()
```

❑ **Use Filters:**

```
session.query(User).filter(User.age > 25)
```

Backend Insight:

- Fetch only required data

2.9 Transactions in SQLAlchemy

```
try:  
    session.add(user)  
    session.commit()
```

```
except:
    session.rollback()
```

Backend Insight:

- Always handle transactions

2.10 Dependency Injection with DB (FastAPI)

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Usage:

```
@app.get("/users")
def get_users(db = Depends(get_db)):
    return db.query(User).all()
```

3. Real-World Examples

Example 1: Create User

```
user = User(name="Ali")  
session.add(user)  
session.commit()
```

Example 2: Fetch Users

```
session.query(User).all()
```

Example 3: Relationship Query

```
user.orders
```

Example 4: Filter Data

```
session.query(User).filter(User.name == "Ali")
```

4. Code Examples (Important Patterns)

Pattern 1: Model

```
class User(Base):
```

Pattern 2: Query

```
session.query(User)
```

Pattern 3: Relationship

```
relationship("Order")
```

Pattern 4: Transaction

```
commit()
```

5. Common Mistakes

✗ Not closing sessions

✗ Fetching unnecessary data

✗ Ignoring relationships

✗ Not handling transactions

✗ N+1 query problem

6. Interview Questions with Answers

Q1: What is ORM?

Mapping between objects and database

Q2: What is SQLAlchemy?

Python ORM

Q3: What is a model?

Class representing table

Q4: What is session?

Database connection

Q5: What is relationship?

Link between tables

Q6: What is lazy loading?

Load data when needed

Q7: What is eager loading?

Load data immediately

Q8: Why use ORM?

Cleaner code

7. Summary (Quick Revision)

- ORM = database abstraction
- SQLAlchemy = popular ORM
- Use:
 - models
 - sessions
 - relationships
- CRUD operations:
 - Create
 - Read
 - Update
 - Delete
- Optimize queries

□ Master SQLAlchemy =

✓ Real backend development

✓ Clean database code

✓ Strong interview performance □

□ **PART 3: NoSQL & Caching**

Chapter 12: MongoDB

(Documents, Collections, CRUD Operations, Aggregation, When to Use NoSQL)

1. Introduction (Why This Matters in Backend)

So far, you learned SQL (relational databases).

But modern backend systems often use **NoSQL databases**.

□ **Why NoSQL?**

Relational databases have limitations:

- Fixed schema
- Difficult scaling
- Complex joins

□ NoSQL solves these problems:

- Flexible schema
- Easy horizontal scaling
- Faster for some use cases

□ What is MongoDB?

MongoDB is a **NoSQL document database**.

□ Data is stored as:

- JSON-like documents

□ Real Backend Use:

- Social media apps
- Real-time systems
- Logs & analytics
- Product catalogs

□ If you master MongoDB:

- You can build modern backend systems
- You understand NoSQL vs SQL
- You perform well in interviews

2. Core Concepts

2.1 SQL vs NoSQL (Important Comparison)

Feature	SQL	NoSQL (MongoDB)
Schema	Fixed	Flexible
Data	Tables	Documents
Scaling	Vertical	Horizontal

Joins Supported Limited

□ **Backend Insight:**

- SQL → structured data
- NoSQL → flexible/unstructured data

2.2 MongoDB Data Model

□ **Database → Collection → Document**

Example:

Database: shop

Collection: users

Document:

```
{  
  "name": "Ali",  
  "age": 25  
}
```

□ **Key Difference:**

SQL:

- Tables + rows

MongoDB:

- Collections + documents

2.3 Installing MongoDB (Quick Setup)

Install:

```
pip install pymongo
```

Connect to MongoDB:

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["test_db"]
```

2.4 CRUD Operations (Very Important)

□ Create (Insert)

```
collection = db["users"]  
  
collection.insert_one({"name": "Ali", "age": 25})
```

□ Read (Find)

```
users = collection.find()  
  
for user in users:  
    print(user)
```

□ Filter Data

```
collection.find({"age": 25})
```

□ Update

```
collection.update_one(  
    {"name": "Ali"},  
    {"$set": {"age": 26}}
```

)

□ Delete

```
collection.delete_one({"name": "Ali"})
```

2.5 Query Operators

Common Operators:

Operator	Meaning
\$gt	Greater than
\$lt	Less than
\$in	In list
\$and	AND condition

Example:

```
collection.find({"age": {"$gt": 20}})
```

2.6 Indexing in MongoDB

□ What is Index?

Improves query speed

Example:

```
collection.create_index("name")
```

Backend Insight:

- Important for large data
- Improves performance

2.7 Aggregation (Advanced)

□ What is Aggregation?

Used for:

- Data processing
- Analytics

Example:

```
pipeline = [  
    {"$match": {"age": {"$gt": 20}}},  
    {"$group": {"_id": "$age", "count": {"$sum": 1}}}  
]  
  
collection.aggregate(pipeline)
```

□ Stages:

Stage	Purpose
\$match	Filter
\$group	Group data
\$sort	Sort
\$project	Select fields

2.8 Embedded vs Referenced Data

□ Embedded (Nested)

```
{  
  "name": "Ali",  
  "orders": [  
    {"product": "Phone"}  
  ]  
}
```

```
}
```

□ Referenced

```
{  
  "user_id": 1  
}
```

Backend Insight:

Type	Use
Embedded	small data
Referenced	large data

2.9 Schema Design in MongoDB

□ Flexible Schema:

```
{  
  "name": "Ali"  
}  
{  
  "name": "Sara",  
  "age": 30  
}
```

}

- Documents can have different fields

Backend Insight:

- Flexibility = power
- But requires careful design

2.10 When to Use MongoDB

Use MongoDB When:

- Data is unstructured
- Rapid development needed
- High scalability required

Avoid MongoDB When:

- Complex joins needed
- Strict schema required

3. Real-World Examples

Example 1: User Collection

```
collection.insert_one({"name": "Ali"})
```

Example 2: Query Data

```
collection.find({"age": {"$gt": 25}})
```

Example 3: Update Data

```
collection.update_one(...)
```

Example 4: Aggregation

```
collection.aggregate(pipeline)
```

4. Code Examples (Backend Patterns)

Pattern 1: Insert

```
insert_one()
```

Pattern 2: Query

```
find()
```

Pattern 3: Update

```
update_one()
```

Pattern 4: Aggregate

```
aggregate()
```

5. Common Mistakes

✗ No indexing

✗ Poor schema design

✗ Overusing embedded documents

✗ Ignoring data size

✗ Treating MongoDB like SQL

6. Interview Questions with Answers

Q1: What is MongoDB?

NoSQL document database

Q2: SQL vs MongoDB?

Tables vs documents

Q3: What is collection?

Group of documents

Q4: What is document?

JSON-like data

Q5: What is aggregation?

Data processing pipeline

Q6: Embedded vs referenced?

Nested vs linked

Q7: When to use MongoDB?

Flexible data

Q8: What is indexing?

Improves query speed

7. Summary (Quick Revision)

- MongoDB = NoSQL database
- Structure:
 - Database
 - Collection
 - Document
- CRUD:
 - insert
 - find
 - update
 - delete
- Aggregation → advanced queries
- Index → performance

□ Master MongoDB =

✓ Modern backend systems

✓ Flexible data handling

✓ Strong interview performance □

Chapter 13: Redis

(Caching, Data Structures, Performance Optimization, Real-World Use Cases)

1. Introduction (Why This Matters in Backend)

In real backend systems, speed is everything.

❑ Problem:

Database queries can be:

- Slow
- Expensive
- Repeated

❑ Example:

User opens homepage:

- Backend fetches same data again and again
- Database gets overloaded

❑ Solution:

❑ Caching using Redis

□ What is Redis?

Redis is an **in-memory data store**:

- Extremely fast
- Key-value based
- Used for caching, sessions, queues

□ Why Redis is Fast?

- Stores data in RAM (not disk)
- No complex joins
- Optimized for speed

□ If you master Redis:

- You can build **high-performance systems**
- You reduce database load
- You pass **system design interviews**

2. Core Concepts

2.1 What is Caching?

Caching means:

- Storing frequently used data for quick access

❑ Without Cache:

Client → Backend → Database → Response

❑ With Cache:

Client → Backend → Redis → Response
 ↓ (if miss)
 Database

❑ First request → DB

❑ Next requests → Redis

2.2 Redis Basics

❑ Key-Value Store

key → value

Example:

"user:1" → {"name": "Ali"}

❑ **Install Redis Client:**

```
pip install redis
```

❑ **Connect to Redis:**

```
import redis  
  
r = redis.Redis(host="localhost", port=6379, db=0)
```

2.3 Basic Operations

❑ **Set Value:**

```
r.set("name", "Ali")
```

❑ **Get Value:**

```
r.get("name")
```

- ❑ **Delete Key:**

```
r.delete("name")
```

- ❑ **Check Exists:**

```
r.exists("name")
```

2.4 Expiration (TTL) — Very Important

- ❑ **What is TTL?**

Time-To-Live:

- ❑ Key expires automatically

Example:

```
r.setex("user:1", 60, "Ali")
```

- ❑ Expires in 60 seconds

Backend Insight:

- Prevent stale data
- Automatically clears cache

2.5 Redis Data Structures

Redis supports multiple data types.

□ 1. String

```
r.set("key", "value")
```

□ 2. List

```
r.lpush("tasks", "task1")
```

□ 3. Set

```
r.sadd("users", "Ali")
```

□ 4. Hash (Very Important)

```
r.hset("user:1", "name", "Ali")
```

Backend Insight:

- Hash used for objects
- Lists used for queues

2.6 Caching Pattern (Very Important)

Example:

```
def get_user(user_id):  
    cached = r.get(f"user:{user_id}")  
  
    if cached:  
        return cached  
  
    data = get_from_db(user_id)  
    r.setex(f"user:{user_id}", 60, data)  
  
    return data
```

□ **Flow:**

1. Check Redis
2. If exists → return
3. Else → fetch DB
4. Store in Redis

2.7 Cache Invalidation (Important)

□ **Problem:**

Data changes in DB but cache still old

□ **Solution:**

```
r.delete(f"user:{user_id}")
```

Backend Insight:

- Always update or delete cache

2.8 Redis for Sessions

Example:

```
r.setex("session:123", 3600, "user_id")
```

Backend Use:

- Store login sessions
- Token storage

2.9 Redis for Rate Limiting

Example:

```
count = r.incr("user:1")  
  
if count > 5:  
    return "Too many requests"
```

Backend Insight:

- Prevent abuse

- Used in APIs

2.10 Redis as Message Queue

Example:

```
r.lpush("queue", "task")
```

Backend Use:

- Background jobs
- Task processing

2.11 When to Use Redis

Use Redis For:

- Caching
- Sessions
- Rate limiting
- Queues

Avoid Redis For:

- Permanent storage
- Large complex queries

3. Real-World Examples

Example 1: Cache User Data

```
r.setex("user:1", 60, "Ali")
```

Example 2: Session Storage

```
r.set("session:1", "user_id")
```

Example 3: Rate Limiting

```
r.incr("user:1")
```

Example 4: Queue

```
r.lpush("tasks", "email")
```

4. Code Examples (Important Patterns)

Pattern 1: Cache

```
r.setex(key, 60, value)
```

Pattern 2: Get Cache

```
r.get(key)
```

Pattern 3: Delete Cache

```
r.delete(key)
```

Pattern 4: Increment

```
r.incr(key)
```

5. Common Mistakes

✗ Not setting TTL

✗ Storing large data

✗ Not invalidating cache

✗ Using Redis as primary DB

✗ Ignoring memory limits

6. Interview Questions with Answers

Q1: What is Redis?

In-memory key-value store

Q2: Why Redis is fast?

Stored in RAM

Q3: What is caching?

Store frequent data

Q4: What is TTL?

Expiration time

Q5: What is cache invalidation?

Removing outdated data

Q6: Redis use cases?

- Cache
- Sessions
- Rate limiting

Q7: What is Redis data structure?

String, list, set, hash

Q8: Redis vs DB?

Redis → fast, temporary

DB → permanent

7. Summary (Quick Revision)

- Redis = in-memory datastore
- Used for:
 - caching
 - sessions
 - rate limiting
- Key features:
 - fast
 - TTL
 - data structures
- Always:
 - set expiration
 - invalidate cache

□ Master Redis =

- ✓ High-performance backend
- ✓ Scalable systems
- ✓ Strong interview advantage ☐

□ **PART 4: Backend Projects**

Chapter 14: Build Production Backend

(FastAPI Project, Auth System, Database Integration, Clean Architecture)

1. Introduction (Why This Matters in Backend)

So far, you have learned:

- FastAPI
- Databases (SQL + MongoDB)
- Redis
- Authentication

□ Now comes the most important part:

□ **Building a Real Backend Project**

□ **Why Projects Matter?**

In interviews:

- Theory is not enough ✕
- Projects prove your skills ✓

□ **What You Will Build:**

A Production-Ready Backend System with:

- User authentication
- Database integration
- Clean architecture
- Scalable structure

□ If you master this:

- You become job-ready
- You can explain projects confidently
- You stand out in interviews

2. Core Concepts

2.1 Project Overview

□ **Example Project:**

□ **User Management System**

Features:

- Register user
- Login user
- Get user profile
- Secure endpoints

□ Tech Stack:

- FastAPI → API
- SQLAlchemy → Database
- Redis → Cache (optional)
- JWT → Authentication

2.2 Project Structure (Very Important)

Recommended Structure:

```
project/
|
├── app/
|   ├── main.py
|   ├── api/
|   |   ├── routes.py
|   ├── models/
|   |   ├── user.py
|   ├── schemas/
|   |   ├── user.py
|   ├── services/
|   |   ├── user_service.py
|   ├── db/
|   |   ├── database.py
|   └── core/
```

| | |— security.py

□ Why Structure Matters?

- Easy to maintain
- Scalable
- Clean separation

2.3 Database Setup

database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine("sqlite:///test.db")

SessionLocal = sessionmaker(bind=engine)
```

Dependency:

```
def get_db():
    db = SessionLocal()
    try:
```

```
        yield db
    finally:
        db.close()
```

2.4 User Model

```
from sqlalchemy import Column, Integer, String
from app.db.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True)
    password = Column(String)
```

2.5 Pydantic Schemas

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    email: str
    password: str
```

2.6 Authentication System

Hash Password:

```
def hash_password(password):  
    return pwd_context.hash(password)
```

Create Token:

```
def create_token(data):  
    return jwt.encode(data, SECRET_KEY)
```

2.7 User Service (Business Logic)

```
def create_user(db, user):  
    hashed = hash_password(user.password)  
  
    new_user = User(  
        email=user.email,  
        password=hashed  
    )  
  
    db.add(new_user)
```

```
db.commit()
return new_user
```

□ Why Service Layer?

- Keeps routes clean
- Separates logic

2.8 API Routes

Register:

```
@app.post("/register")
def register(user: UserCreate, db=Depends(get_db)):
    return create_user(db, user)
```

Login:

```
@app.post("/login")
def login():
    return {"token": "abc"}
```

Protected Route:

```
@app.get("/profile")
def profile(user=Depends(get_current_user)):
    return user
```

2.9 Clean Architecture (Important)

Layers:

1. Routes → API layer
2. Services → business logic
3. Models → database
4. Schemas → validation

Backend Insight:

- Never mix all logic in one place

2.10 Error Handling

```
from fastapi import HTTPException

if not user:
    raise HTTPException(status_code=404)
```

2.11 Adding Redis (Optional)

```
r.setex("user:1", 60, "Ali")
```

Use:

- Cache user data
- Improve performance

2.12 Production Best Practices

☐ **Use Environment Variables**

☐ **Use Logging**

❑ **Validate Inputs**

❑ **Use HTTPS**

❑ **Proper Error Handling**

3. Real-World Example (Flow)

User Registration Flow:

1. User sends data
2. Backend validates
3. Password hashed
4. Stored in DB

Login Flow:

1. Verify password
2. Generate JWT
3. Return token

Protected Route Flow:

1. Receive token
2. Validate token
3. Return data

4. Code Examples (Important Patterns)

Pattern 1: Dependency Injection

```
db = Depends(get_db)
```

Pattern 2: Service Layer

```
create_user(db, user)
```

Pattern 3: Authentication

```
get_current_user()
```

Pattern 4: Response Format

```
return {"status": "success"}
```

5. Common Mistakes

✗ Mixing business logic in routes

✗ Not hashing passwords

✗ Poor project structure

✗ No validation

✗ No authentication

6. Interview Questions with Answers

Q1: How to structure backend project?

Use layered architecture

Q2: What is service layer?

Business logic layer

Q3: How to secure API?

JWT authentication

Q4: How to connect DB?

Using SQLAlchemy

Q5: What is dependency injection?

Inject DB/session

Q6: Why use Pydantic?

Validation

Q7: What is clean architecture?

Separation of concerns

Q8: What makes backend production-ready?

- Security

- Structure
- Performance

7. Summary (Quick Revision)

- Build real backend project
- Use:
 - FastAPI
 - SQLAlchemy
 - JWT
- Follow structure:
 - routes
 - services
 - models
- Always:
 - validate input
 - hash passwords
 - use clean architecture

☐ Master this chapter =

✓ Real backend development

✓ Strong portfolio project

✓ Interview-ready ☐

Chapter 15: Optimization

(Caching, Query Optimization, Performance Tuning, Scaling Basics)

1. Introduction (Why This Matters in Backend)

Building a backend is not enough.

- ❑ It must be **fast, efficient, and scalable**.

❑ Real Problems in Backend Systems:

- Slow APIs
- High database load
- Server crashes under traffic
- Poor user experience

❑ Example:

Without optimization:

- API takes 3 seconds ✗

With optimization:

- API takes 100ms ✓

❑ This is the difference between:

- Beginner backend ✗
- Production backend ✓

☐ If you master optimization:

- Your systems handle real users
- You impress interviewers
- You build scalable applications

2. Core Concepts

2.1 What is Optimization?

Optimization means:

☐ Improving performance and efficiency

☐ **Areas of Optimization:**

- Database queries
- API response time
- Memory usage
- Network calls

2.2 Caching (Very Important)

□ Why Caching?

Repeated queries slow down system

□ Solution:

Store result in Redis

Example:

```
def get_user(user_id):  
    cached = r.get(f"user:{user_id}")  
  
    if cached:  
        return cached  
  
    user = db.query(User).get(user_id)  
  
    r.setex(f"user:{user_id}", 60, user)  
    return user
```

□ Benefits:

- Faster response
- Reduced DB load

2.3 Database Query Optimization

❑ Problem:

```
SELECT * FROM users;
```

❑ Solution:

```
SELECT name FROM users WHERE age > 25;
```

❑ Best Practices:

- Avoid SELECT *
- Use WHERE clause
- Use indexes

❑ Index Example:

```
CREATE INDEX idx_age ON users(age);
```

2.4 N+1 Query Problem

❑ What is N+1?

```
users = db.query(User).all()
```

```
for user in users:  
    print(user.orders)
```

❑ Many queries executed

❑ Solution:

```
from sqlalchemy.orm import joinedload
```

```
db.query(User).options(joinedload(User.orders))
```

Backend Insight:

- Common performance issue

2.5 API Optimization

❑ **Reduce Response Size**

```
return {"name": user.name}
```

❑ **Use Pagination**

```
@app.get("/users")
def get_users(limit: int = 10):
    return users[:limit]
```

❑ **Use Compression (Concept)**

- Reduce payload size

2.6 Async Programming (Performance Boost)

Example:

```
@app.get("/")
async def home():
    return {"message": "fast"}
```

□ **Why Async?**

- Handles multiple requests
- Improves throughput

Backend Insight:

- Best for I/O operations

2.7 Connection Pooling

□ **What is Connection Pool?**

Reuse database connections

Benefits:

- Faster DB access
- Reduced overhead

Backend Insight:

- Used in production systems

2.8 Load Balancing (Basic Concept)

□ What is Load Balancing?

Distribute traffic across servers

Example:

User → Load Balancer → Server 1 / Server 2

Backend Benefit:

- Prevent overload
- Improve reliability

2.9 Horizontal vs Vertical Scaling

□ Vertical Scaling:

- Increase server power

□ **Horizontal Scaling:**

- Add more servers

Backend Insight:

- Modern systems use horizontal scaling

2.10 Monitoring & Logging

□ **Why Monitor?**

- Detect slow queries
- Identify issues

Example:

```
print("Request took X ms")
```

Backend Insight:

- Essential for production

3. Real-World Examples

Example 1: Cache Data

```
r.setex("user:1", 60, user)
```

Example 2: Optimize Query

```
SELECT name FROM users;
```

Example 3: Async API

```
async def route():
```

Example 4: Pagination

```
limit=10
```

4. Code Examples (Important Patterns)

Pattern 1: Cache First

```
if cached:  
    return cached
```

Pattern 2: Efficient Query

```
SELECT column FROM table;
```

Pattern 3: Async Function

```
async def function():
```

Pattern 4: Pagination

limit, offset

5. Common Mistakes

✗ Not using caching

✗ Using SELECT *

✗ No pagination

✗ Ignoring N+1 problem

✗ No monitoring

6. Interview Questions with Answers

Q1: What is caching?

Store data for faster access

Q2: What is N+1 problem?

Multiple unnecessary queries

Q3: How to optimize queries?

Indexes + filtering

Q4: What is connection pooling?

Reusing DB connections

Q5: Async vs sync?

Async handles multiple requests

Q6: What is load balancing?

Distribute traffic

Q7: Horizontal vs vertical scaling?

More servers vs bigger server

Q8: Why pagination?

Limit data

7. Summary (Quick Revision)

- Optimization = performance improvement
 - Use:
 - caching
 - indexing
 - async
 - Avoid:
 - SELECT *
 - N+1 queries
 - Scale using:
 - load balancing
 - horizontal scaling
- Master optimization =
- ✓ High-performance backend
- ✓ Scalable systems
- ✓ Strong interview advantage □

